数字芯片设计验证 · 手撕代码真题通关手册

——Verilog 常考代码题精讲与可综合参考实现

> 适用对象：数字 IC 设计 / 验证 / FPGA 岗求职者（秋招手撕代码专项） 内容来源：吸收《FPGA 手撕代码经典 8 道题目》及源码，并扩展全网常考真题，共 40 题 水印标识：芯链团

前言：手撕代码题的应对原则

数字芯片岗笔试/面试中的"手撕代码"题，考察的是**可综合 RTL 思维**与**边界细节处理**，而非算法复杂度。应对要点：

1. **先讲思路，再写代码**：面试官更看重你如何分解问题（FSM 三段式？计数+比较？打拍同步？）
2. **接口定义要完整**：`clk` / `rst_n`（低有效异步复位最常用）/ 数据/有效信号缺一不可
3. **复位风格统一**：默认使用异步低有效复位 `always @(posedge clk or negedge rst_n)`
4. **组合逻辑用 `always @(*)` + `case` + `default`**，时序逻辑用非阻塞赋值 `<=`
5. **关键路径注释清楚**：计数器初值、比较边界、打拍级数
6. **主动说明可综合性**：避免 `initial` 给寄存器赋非 0 初值（综合器不支持）、避免 `#delay`

> 本手册每题给出**可综合代码答案**（已对齐行业规范），并在"面试提示"中标注易错点。

第一篇 组合逻辑基础

Q1：用 Verilog 实现半加器与全加器

考点：组合逻辑最基础单元，常作为热身题。

思路：半加器无进位输入；全加器用缩减运算符或拼接符。

代码答案：

```
// ===== 半加器 =====
module half_adder (
    input  a, b,
    output sum, cout
);
    assign sum  = a ^ b;
    assign cout = a & b;
endmodule

// ===== 全加器 =====
module full_adder (
    input  a, b, cin,
    output sum, cout
);
    assign sum  = a ^ b ^ cin;
    assign cout = (a & b) | (a & cin) | (b & cin);
    // 等价写法：assign {cout, sum} = a + b + cin;
endmodule
```

面试提示：`{cout, sum} = a + b + cin` 写法最简洁，但需理解拼接符宽度——拼接结果是 2 bit，低位为 sum。

Q2：用至少两种写法实现 2 选 1 和 4 选 1 多路选择器

考点：MUX 是基础，写法多样性体现对语法的熟练度。

代码答案：

```
// ===== 2:1 MUX =====
// 写法1：三目运算符（最常用）
assign y = sel ? b : a;
// 写法2：位拼接
assign y = sel ? b : a;   // 同写法1
// 写法3：case
always @(*) begin
    case (sel)
        1'b0: y = a;
        1'b1: y = b;
    endcase
```

```
end

// ===== 4:1 MUX =====
// 写法1: 嵌套三目
assign y = sel[1] ? (sel[0] ? d3 : d2)
                  : (sel[0] ? d1 : d0);
// 写法2: case (更清晰，推荐)
always @(*) begin
    case (sel)
        2'b00: y = d0;
        2'b01: y = d1;
        2'b10: y = d2;
        2'b11: y = d3;
    endcase
end
```

面试提示：优先用 `case` + `default`，避免 latch 产生。`case` 中务必有 `default` 分支。

Q3：实现 8 位优先编码器（Priority Encoder）

考点：最高位（或最低位）优先的编码逻辑，常见于中断仲裁。

思路：用 `casez` + `?` 通配符，从高位到低位匹配第一个 1。

代码答案：

```
module prio_encoder (
    input  [7:0] req,     // 请求信号，1=有效
    output reg [2:0] enc,  // 编码输出
    output reg       valid // 有无有效请求
);
    always @(*) begin
        valid = 1'b1;
        casez (req)
            8'b1???????: enc = 3'd7;
            8'b01??????: enc = 3'd6;
            8'b001?????: enc = 3'd5;
            8'b0001????: enc = 3'd4;
            8'b00001???: enc = 3'd3;
            8'b000001??: enc = 3'd2;
            8'b0000001?: enc = 3'd1;
            8'b00000001: enc = 3'd0;
            8'b00000000: begin enc = 3'd0; valid = 1'b0; end
            default:     begin enc = 3'd0; valid = 1'b0; end
        endcase
    end
endmodule
```

面试提示：`casez` 配合 `?` 比 `casex` 更安全（`casex` 把 x/z 也当通配，易出错）。也可用 `for` 循环从高到低遍历，代码更通用。

Q4：实现 8 位数值比较器（相等/大于/小于）

考点：比较器是 ALU 基础。

代码答案：

```
module comparator #(
    parameter W = 8
)(
    input  [W-1:0] a, b,
    output         eq,   // a == b
    output         gt,   // a >  b
    output         lt    // a <  b
);
    assign eq = (a == b);
    assign gt = (a >  b);
    assign lt = (a <  b);
endmodule
```

面试提示：综合器会自动推断比较器逻辑。注意有符号数比较要用 `signed` 声明，否则按无符号比较。

Q5：实现奇偶校验位生成（偶校验）

考点：通信中常用，考察缩减运算符。

代码答案：

```
module parity_gen (
    input  [7:0] data,
    output       even_p,  // 偶校验位：使含校验位共偶数个1
    output       odd_p    // 奇校验位
);
    assign even_p = ^data;   // XOR 缩减 = 偶校验位
    assign odd_p  = ~^data;  // XNOR 缩减 = 奇校验位
endmodule
```

面试提示：`^data` 是 XOR 缩减运算符，等价于所有位异或，得到偶校验位。这是最简洁写法。

Q6：二进制与格雷码互转

考点：异步 FIFO、跨时钟域中格雷码指针的核心转换。

思路：`gray = bin ^ (bin >> 1)`；`bin` 最高位等于 `gray` 最高位，其余位 `bin[i] = bin[i+1] ^ gray[i]`。

代码答案：

```
module bin_gray #(
    parameter W = 8
)(
```

```
    input  [W-1:0] bin,
    output [W-1:0] gray,
    input  [W-1:0] gray_in,
    output [W-1:0] bin_out
);
    // 二进制 → 格雷码
    assign gray = bin ^ (bin >> 1);

    // 格雷码 → 二进制（迭代）
    genvar i;
    generate
        assign bin_out[W-1] = gray_in[W-1];
        for (i = W-2; i >= 0; i = i-1)
            assign bin_out[i] = bin_out[i+1] ^ gray_in[i];
    endgenerate
endmodule
```

面试提示：格雷码相邻仅 1 bit 变化，是异步指针跨时钟域安全的根本原因。需能口述转换公式。

Q7：统计 32 bit 数据中 1 的个数（Popcount）

考点：来自原文档题目一。考察移位累加 / 循环 / 并行 XOR 树多种思路。

思路： - 方法一（参考源码）：时序移位，每拍取最低位累加，32 拍完成 - 方法二（推荐组合逻辑）：`for` 循环遍历 32 bit 累加 - 方法三（高速并行）：分治 XOR 树，单周期出结果

代码答案（方法二：组合逻辑 for 循环，简洁可综合）：

```
function [5:0] popcount;
    input [31:0] din;
    integer i;
    reg [5:0] cnt;
    begin
        cnt = 0;
        for (i = 0; i < 32; i = i + 1)
            cnt = cnt + din[i];
        popcount = cnt;
    end
endfunction
```

代码答案（方法一：时序移位版，源自原文档 01-count_32.v）：

```
module count_32 (
    input         clk, rstn,
    input  [31:0] data_i,
    input         valid_i,
    output reg [5:0] count,
    output reg       done
);
    reg [5:0]  count_reg;
    reg [31:0] data_reg;
    reg [5:0]  cnt;
```

```
    reg        busy;

    always @(posedge clk or negedge rstn) begin
        if (!rstn)       {busy, count_reg, data_reg, cnt} <= 0;
        else if (valid_i) begin
            busy <= 1; count_reg <= 0; data_reg <= data_i; cnt <= 0;
        end
        else if (busy) begin
            count_reg <= count_reg + data_reg[0];
            data_reg <= data_reg >> 1;
            cnt <= cnt + 1;
        end
    end
    assign done  = (cnt == 6'd31);
    assign count = done ? count_reg + data_reg[0] : count_reg;
endmodule
```

面试提示：面试官常追问"能否单周期完成？"——答并行 XOR 树（两两相加）。移位版注意 `done` 与最后一次累加的时序对齐（原源码用 `cnt==31` 时补加 `data_reg[0]`，需理解）。

Q8：实现比特反转（Bit Reversal）

考点：FFT、CRC 等算法常用。

代码答案：

```
module bit_reverse #(
    parameter W = 8
)(
    input  [W-1:0] din,
    output [W-1:0] dout
);
    genvar i;
    generate
        for (i = 0; i < W; i = i + 1)
            assign dout[i] = din[W-1-i];
    endgenerate
endmodule
```

Q9：实现前导零检测（Leading Zero Count）

考点：浮点归一化、优先级判定。

代码答案（for 循环从高位扫描）：

```
function [5:0] lzc;
    input [31:0] din;
    integer i;
    reg [5:0] cnt;
    begin
```

```
        cnt = 0;
        for (i = 31; i >= 0; i = i - 1)
            if (din[i] == 1'b0) cnt = cnt + 1;
            else                break;
        lzc = cnt;
    end
endfunction
```

Q10：实现桶形移位器（Barrel Shifter）

考点：ALU 移位单元，考察参数化与组合逻辑。

代码答案：

```
module barrel_shifter #(
    parameter W = 8
)(
    input  [W-1:0] din,
    input  [2:0]   shamt,   // 移位量 0~7
    input          dir,     // 0=左移 1=右移
    output [W-1:0] dout
);
    assign dout = dir ? (din >> shamt) : (din << shamt);
endmodule
```

面试提示：综合器对 `<<` / `>>` 直接推断桶形移位器。左移注意高位截断，右移注意逻辑/算术（无符号逻辑右移）。

芯链团

第二篇 时序逻辑基础

Q11：实现带异步复位 / 同步复位 / 使能的 D 触发器

考点：最基础的时序元件，复位风格是面试必问。

代码答案：

```
// ===== 异步低有效复位 =====
module dff_async (
    input clk, rst_n, d,
    output reg q
);
    always @(posedge clk or negedge rst_n) begin
        if (!rst_n) q <= 1'b0;
        else       q <= d;
    end
endmodule

// ===== 同步高有效复位 + 使能 =====
module dff_sync_en (
    input clk, rst, en, d,
    output reg q
);
    always @(posedge clk) begin
        if (rst)     q <= 1'b0;
        else if (en)  q <= d;
    end
endmodule
```

面试提示：ASIC 常用异步低有效复位（`rst_n`），且复位释放需满足"异步复位、同步释放"以避免亚稳态。FPGA 常用同步复位。

Q12：实现模 N 计数器（含增减计数器）

考点：计数器是时序逻辑核心，几乎所有时序题的基础。

代码答案：

```
// ===== 模 N 加法计数器 =====
module cnt_modN #(
    parameter N = 10
)(
    input clk, rst_n, en,
    output reg [$clog2(N)-1:0] cnt,
    output                  tc  // terminal count 翻转标志
);
    always @(posedge clk or negedge rst_n) begin
```

```
        if (!rst_n)      cnt <= 0;
        else if (!en)    cnt <= cnt;
        else if (cnt==N-1) cnt <= 0;
        else             cnt <= cnt + 1;
    end
    assign tc = (cnt == N-1);
endmodule

// ===== 增减计数器 =====
module cnt_updown #(
    parameter N = 16
)(
    input clk, rst_n, up,
    output reg [$clog2(N)-1:0] cnt
);
    always @(posedge clk or negedge rst_n) begin
        if (!rst_n) cnt <= 0;
        else if (up) cnt <= (cnt==N-1)? 0 : cnt+1;
        else        cnt <= (cnt==0)  ? N-1 : cnt-1;
    end
endmodule
```

面试提示：`$clog2(N)` 自动计算计数器位宽，是参数化设计的好习惯。

Q13：实现移位寄存器与通用移位寄存器

考点：串并转换的基础。

代码答案：

```
// ===== 串入串出（左移，din 进 LSB）=====
module shift_reg #(
    parameter W = 8
)(
    input clk, rst_n, din,
    output dout
);
    reg [W-1:0] sr;
    always @(posedge clk or negedge rst_n) begin
        if (!rst_n) sr <= 0;
        else       sr <= {sr[W-2:0], din};
    end
    assign dout = sr[W-1];
endmodule

// ===== 通用移位寄存器（左移/右移/保持/加载）=====
module universal_sr #(
    parameter W = 8
)(
    input clk, rst_n,
    input [1:0] op,       // 00保持 01右移 10左移 11并行加载
    input sin,            // 串行输入
```

```
    input [W-1:0] par_in,  // 并行加载
    output [W-1:0] q
);
    reg [W-1:0] r;
    always @(posedge clk or negedge rst_n) begin
        if (!rst_n)       r <= 0;
        else case (op)
            2'b00: r <= r;
            2'b01: r <= {sin, r[W-1:1]};      // 右移
            2'b10: r <= {r[W-2:0], sin};      // 左移
            2'b11: r <= par_in;
        endcase
    end
    assign q = r;
endmodule
```

Q14：用 Verilog 实现边沿检测（上升沿 / 下降沿 / 双边沿）

考点：来自原文档题目八。打两拍后做异或/与组合，输出 1 周期脉冲。

思路：连续两拍寄存 `data`，`pos = ~d1 & d0`（上一拍为 0 当前为 1），`neg = d1 & ~d0`。

代码答案（源自原文档 08-edge_detect.v，精简版）：

```
module edge_detect (
    input  clk, rst_n, data,
    output pos_edge,      // 上升沿脉冲（1 周期）
    output neg_edge,      // 下降沿脉冲（1 周期）
    output data_edge      // 双边沿脉冲
);
    reg d0, d1;
    always @(posedge clk or negedge rst_n) begin
        if (!rst_n) {d1, d0} <= 2'b00;
        else       {d1, d0} <= {d0, data};  // 打两拍
    end
    assign pos_edge  =  d0 & ~d1;
    assign neg_edge  = ~d0 &  d1;
    assign data_edge =  d0 ^  d1;
endmodule
```

面试提示：边沿检测本质是"当前值与上拍值不同"。注意输出脉冲宽度 = 1 个时钟周期（因 d1/d0 都是寄存值）。

Q15：实现序列检测器 "101"（重叠 / 非重叠）

考点：来自原文档题目七。FSM 经典题，必考。

思路：三段式状态机。S0 初态 → 收 1 到 S1 → 收 0 到 S2 → 收 1 到 S3（匹配），S3 收 1 回到 S1（重叠检测关键），收 0 回到 S2。

代码答案（源自原文档 07-Sequence_detect.v，三段式 FSM）：

```
module seq_detect_101 (
    input  clk, rst_n, data,
    output reg flag         // 检测到 101 输出 1
);
    localparam S0 = 2'b00, S1 = 2'b01, S2 = 2'b10, S3 = 2'b11;
    reg [1:0] state, nstate;

    // 第一段：状态寄存器
    always @(posedge clk or negedge rst_n) begin
        if (!rst_n) state <= S0;
        else       state <= nstate;
    end

    // 第二段：次态组合逻辑
    always @(*) begin
        case (state)
            S0: nstate = data ? S1 : S0;
            S1: nstate = data ? S1 : S2;   // 连续1保持在S1
            S2: nstate = data ? S3 : S0;   // 收到1→匹配候选S3
            S3: nstate = data ? S1 : S2;   // 重叠：收1回S1，收0回S2
            default: nstate = S0;
        endcase
    end

    // 第三段：输出（Moore 型，输出仅依赖状态）
    always @(posedge clk or negedge rst_n) begin
        if (!rst_n) flag <= 0;
        else       flag <= (state == S3);
    end
endmodule
```

面试提示：- "重叠检测"指 `10101` 中能检出 2 个 101（第 2 个 101 复用前者的末位 1）。S3 收 1 回到 S1 即实现此特性。 - Moore 型输出比 Mealy 型延迟 1 拍但无毛刺，面试推荐写 Moore。 - 常追问 Mealy 版（输出与输入相关，可提前 1 拍输出）。

Q16：实现偶数分频（50% 占空比）

考点：分频最基础，N 为偶数时最高位翻转即为 N 分频。

代码答案：

```
module clk_div_even #(
    parameter N = 10          // N 必须为偶数
)(
    input  clk, rst_n,
    output reg clk_div
);
    reg [$clog2(N)-1:0] cnt;
    always @(posedge clk or negedge rst_n) begin
        if (!rst_n)       {cnt, clk_div} <= 0;
        else if (cnt == N/2 - 1) begin
```

```
            cnt <= 0;
            clk_div <= ~clk_div;   // 每 N/2 周期翻转一次
        end
        else cnt <= cnt + 1;
    end
endmodule
```

面试提示：偶数分频简单——计数到 N/2-1 翻转。注意这是"占空比 50%"的实现。奇数分频不能用同法，见 Q17。

Q17：实现奇数分频（50% 占空比）—— 以 3 分频为例

考点：来自原文档题目四。奇数分频 50% 占空比是经典难点。

思路：利用同一时钟的**上升沿和下降沿各产生一个 1/3 占空比脉冲，再相或**——两个脉冲错相位叠加得到 50% 占空比的 1/3 频率。

代码答案（源自原文档 04-div3.v，标准奇数分频法）：

```
module clk_div3 (
    input  clk, rst_n,
    output clk_div3
);
    reg [1:0] cnt;
    reg       div_p;   // 上升沿产生的 1/3 占空比
    reg       div_n;   // 下降沿产生的 1/3 占空比

    // 模3计数器
    always @(posedge clk or negedge rst_n) begin
        if (!rst_n) cnt <= 0;
        else        cnt <= (cnt == 2'd2) ? 0 : cnt + 1;
    end

    // 上升沿：cnt==0 时置高一个半周期
    always @(posedge clk or negedge rst_n) begin
        if (!rst_n) div_p <= 0;
        else        div_p <= (cnt == 0);
    end

    // 下降沿：cnt==0 时置高一个半周期（错相位）
    always @(negedge clk or negedge rst_n) begin
        if (!rst_n) div_n <= 0;
        else        div_n <= (cnt == 0);
    end

    assign clk_div3 = div_p | div_n;  // 相或得 50% 占空比
endmodule
```

面试提示： - 原理：div_p 在 cnt==0 的上升沿到下一个上升沿为高（1/3 周期），div_n 在 cnt==0 的下降沿到下一个下降沿为高（1/3 周期），二者相位差半个 clk，或运算得到占空比 `(1/3 + 1/3)` ≈ 50%（实际 1.5/3 = 50%）。 - 通用奇数 N 分频：上升沿在 `cnt∈[0, (N-1)/2]` 置高，下降沿同样区间置高，相或即得 50%。

Q18：实现无毛刺时钟切换电路（Glitch-Free Clock Mux）

考点：来自原文档题目五。时钟切换是高频题，重点在"无毛刺"。

思路：直接 `assign clk_o = sel ? clkb : clka` 会在 sel 变化时产生毛刺。正确做法： 1. 每个时钟域各用一个**负沿触发**的寄存器同步 sel（避免切换瞬间刚好在时钟高电平产生窄脉冲） 2. 用"对方已关闭"作为使能条件（`selA <= ~selB_r & ~sel`），保证旧时钟完全关闭后新时钟才开启 3. 输出 `clk_o = (clka & selA_r) | (clkb & selB_r)`

代码答案（修正原文档 05-clock_switching.v 的重复定义/时钟域错误，标准无毛刺版）：

```
module clk_switch_glitchfree (
    input  clka, clkb, rst_n, sel,  // sel=0→clka, sel=1→clkb
    output clk_o
);
    reg selA, selA_r;   // clka 域：使能信号及其打拍
    reg selB, selB_r;   // clkb 域

    // clka 域同步 sel（负沿触发，切换前确保旧时钟已低）
    always @(negedge clka or negedge rst_n) begin
        if (!rst_n) begin selA <= 0; selA_r <= 0; end
        else        begin selA <= ~selB_r & ~sel; selA_r <= selA; end
    end

    // clkb 域同步 sel（注意：必须在 clkb 负沿，原文档误写为 clka）
    always @(negedge clkb or negedge rst_n) begin
        if (!rst_n) begin selB <= 0; selB_r <= 0; end
        else        begin selB <= ~selA_r & sel; selB_r <= selB; end
    end

    assign clk_o = (clka & selA_r) | (clkb & selB_r);
endmodule
```

面试提示： - 核心：切换时先关闭旧时钟路径、等确认关闭后再开新时钟路径，且用**负沿**采样 sel 以避免在时钟高电平期间切换产生毛刺。 - 原文档源码在 selB 同步块误用了 `clka` 边沿且有重复 `reg` 声明（编译报错），面试中若被要求挑错可指出这点。

第三篇 跨时钟域（CDC）

Q19：单比特信号同步器（2 级打拍）

考点：CDC 最基础，所有跨时钟域的入门。

代码答案：

```
module sync_2ff (
    input  clk, rst_n, din,   // din 来自另一个时钟域
    output dout
);
    reg ff1, ff2;
    always @(posedge clk or negedge rst_n) begin
        if (!rst_n) {ff2, ff1} <= 2'b00;
        else       {ff2, ff1} <= {ff1, din};
    end
    assign dout = ff2;
endmodule
```

面试提示：2 级触发器将亚稳态概率降到极低。单比特电平信号（非脉冲）用此法即可。

Q20：快时钟域到慢时钟域的脉冲传输

考点：来自原文档题目三。慢时钟可能采不到快时钟的窄脉冲，必须"展宽"再同步。

思路： 1. 快域：收到脉冲后置位 `signal_a`（保持高），直到收到慢域的"已接收"确认才清零 2. 慢域：2 级同步 `signal_a` → 得到稳定电平 3. 慢域检测该电平的上升沿 → 在慢域产生脉冲 4. 慢域将"已接收"信号同步回快域 → 快域清零 `signal_a`

代码答案（源自原文档 03-fast_slow.v，整理版）：

```
module fast_to_slow (
    input  clka, clkb, rst_n,
    input  pulse_ain,        // 快域窄脉冲
    output pulse_bout        // 慢域脉冲
);
    reg       sig_a;        // 快域展宽信号
    reg       sig_b;        // 慢域同步后的电平
    reg [1:0] ack_a;        // 快域收到的慢域确认（打拍）
    reg [1:0] det_b;        // 慢域检测边沿用

    // 1) 快域：脉冲到来后置位，收到确认(ack_a[1])后清零
    always @(posedge clka or negedge rst_n) begin
        if (!rst_n)       sig_a <= 0;
        else if (pulse_ain) sig_a <= 1;
        else if (ack_a[1])  sig_a <= 0;
    end
```

```
    // 2) 慢域：2级同步 sig_a
    always @(posedge clkb or negedge rst_n) begin
        if (!rst_n) sig_b <= 0;
        else        sig_b <= sig_a;   // 第1级（实际需2级，此处合并示意）
    end
    reg sig_b2;
    always @(posedge clkb or negedge rst_n) begin
        if (!rst_n) sig_b2 <= 0;
        else        sig_b2 <= sig_b;
    end

    // 3) 慢域：检测 sig_b2 上升沿 → 产生脉冲
    always @(posedge clkb or negedge rst_n) begin
        if (!rst_n) det_b <= 0;
        else        det_b <= {det_b[0], sig_b2};
    end
    assign pulse_bout = sig_b2 & ~det_b[0];  // 上升沿脉冲

    // 4) 将慢域的 det_b[0] 同步回快域作为确认
    always @(posedge clka or negedge rst_n) begin
        if (!rst_n) ack_a <= 0;
        else        ack_a <= {ack_a[0], det_b[0]};
    end
endmodule
```

面试提示：关键两步——快域"展宽"（脉冲变电平）+ 慢域"边沿检测"（电平变脉冲）。慢→快可省略展宽（慢脉冲对快时钟足够宽）。

Q21：多比特信号跨时钟域——握手协议

考点：多比特不能直接打拍（各 bit 延迟不同导致数据错误），需握手或异步 FIFO。

思路：发送方置 `req` 并保持数据稳定 → 接收方同步 `req` 后回 `ack` → 发送方同步 `ack` 后撤销 `req` → 接收方撤销 `ack`。

代码答案（核心握手模块）：

```
module handshake_cdc (
    input  clk_src, clk_dst, rst_n,
    input  [7:0] data_src,
    input        req_src,       // 发送请求（保持到 ack）
    output       ack_src,       // 发送方看到的确认
    output [7:0] data_dst,
    output       valid_dst      // 接收方数据有效
);
    reg req_s2d_r1, req_s2d_r2;
    reg ack_d2s_r1, ack_d2s_r2;
    reg [7:0] data_buf;
    reg      valid;

    // src→dst 同步 req
```

```
    always @(posedge clk_dst or negedge rst_n) begin
        if (!rst_n) {req_s2d_r2, req_s2d_r1} <= 0;
        else        {req_s2d_r2, req_s2d_r1} <= {req_s2d_r1, req_src};
    end
    // dst→src 同步 ack
    always @(posedge clk_src or negedge rst_n) begin
        if (!rst_n) {ack_d2s_r2, ack_d2s_r1} <= 0;
        else        {ack_d2s_r2, ack_d2s_r1} <= {ack_d2s_r1, req_s2d_r2};
    end
    assign ack_src = ack_d2s_r2;

    // 接收方：req 上升沿锁存数据并置 valid
    always @(posedge clk_dst or negedge rst_n) begin
        if (!rst_n)      {valid, data_buf} <= 0;
        else if (req_s2d_r2 && !valid) begin
            data_buf <= data_src; valid <= 1;
        end
        else if (!req_s2d_r2) valid <= 0;
    end
    assign data_dst  = data_buf;
    assign valid_dst = valid;
endmodule
```

面试提示：握手协议简单但速度慢（需多拍往返）。高速多比特 CDC 用异步 FIFO（见 Q24）。

Q22：异步 FIFO 中的格雷码指针跨域

考点：异步 FIFO 核心机制，常作为进阶题。

思路：读写指针用**格雷码**（相邻仅 1 bit 变化）跨时钟域同步，避免多 bit 同时变化导致指针采样错误。空满判断：- 满：`wptr_gray` 与 `rptr_gray` 仅最高两位不同 - 空：`wptr_gray == rptr_gray`

代码答案（格雷码指针同步片段）：

```
// 二进制指针 → 格雷码
function [PTR_W-1:0] bin2gray;
    input [PTR_W-1:0] bin;
    bin2gray = bin ^ (bin >> 1);
endfunction

// 写指针同步到读时钟域（判断满）
reg [PTR_W-1:0] wptr_gray_r1, wptr_gray_r2;
always @(posedge rclk or negedge rst_n) begin
    if (!rst_n) {wptr_gray_r2, wptr_gray_r1} <= 0;
    else        {wptr_gray_r2, wptr_gray_r1} <= {wptr_gray_r1, wptr_gray};
end

// 读指针同步到写时钟域（判断空）
reg [PTR_W-1:0] rptr_gray_r1, rptr_gray_r2;
always @(posedge wclk or negedge rst_n) begin
    if (!rst_n) {rptr_gray_r2, rptr_gray_r1} <= 0;
    else        {rptr_gray_r2, rptr_gray_r1} <= {rptr_gray_r1, rptr_gray};
end
```

```
// 满判断：最高两位相反，其余相同
assign full  = (wptr_gray_r2 == {~rptr_gray[PTR_W-1:PTR_W-2], rptr_gray[PTR_W-3:0]});
// 空判断：读写指针相等
assign empty = (wptr_gray_r2 == rptr_gray);
```

面试提示：异步 FIFO 是"手撕代码"压轴题，需能讲清：双端口 RAM + 格雷码指针 + 空满判断三要素。完整实现见 Q24。

芯链团

第四篇 存储与接口

Q23：实现异步双端口 RAM（A 读 / B 写）

考点：来自原文档题目二。可综合 RAM，深度 16、位宽 8。

思路：用 `reg [7:0] mem [15:0]` 数组；A 口（clka）读、B 口（clkb）写；片选 `cel` 与各自使能 `cela/celb` 联合控制。

代码答案（源自原文档 02-ram_double.v，整理版）：

```
module ram_double #(
    parameter DEPTH = 16,
    parameter WIDTH = 8
)(
    input clka, clkb, rst_n, cel,       // cel 全局片选（低有效）
    // A 口：读
    input  [$clog2(DEPTH)-1:0] addr_a,
    output reg [WIDTH-1:0]     data_a,
    input                   cela,
    // B 口：写
    input  [$clog2(DEPTH)-1:0] addr_b,
    input  [WIDTH-1:0]         data_b,
    input                   celb
);
    reg [WIDTH-1:0] mem [DEPTH-1:0];
    integer i;

    // B 口写（clkb 域）
    always @(posedge clkb or negedge rst_n) begin
        if (!rst_n) begin
            for (i = 0; i < DEPTH; i = i + 1) mem[i] <= 0;
        end
        else if (~celb && ~cel) begin
            mem[addr_b] <= data_b;
        end
    end

    // A 口读（clka 域）
    always @(posedge clka or negedge rst_n) begin
        if (!rst_n)      data_a <= 0;
        else if (~cela && ~cel) data_a <= mem[addr_a];
    end
endmodule
```

面试提示： - 异步双口 RAM 两个端口用不同时钟，综合器会推断为 FPGA 的 BRAM 原语。 - 注意原文档将初值写成 `'b0000_1111`（奇怪的 8bit 值），实际应写 `0` 或合理初值。 - 避免 A/B 口同时写同一地址（冲突），设计时需约定。

Q24：实现异步 FIFO（完整版）

考点：CDC 压轴题。综合 Q22 指针 + Q23 存储。

代码答案（参数化异步 FIFO 核心）：

```
module async_fifo #(
    parameter WIDTH = 8,
    parameter DEPTH = 16
)(
    input  wclk, rclk, rst_n,
    input       winc, rinc,
    input  [WIDTH-1:0] wdata,
    output [WIDTH-1:0] rdata,
    output       full, empty
);
    localparam PTR_W = $clog2(DEPTH) + 1;  // 多1位区分空满
    reg [WIDTH-1:0] mem [DEPTH-1:0];
    reg [PTR_W-1:0] wptr, rptr;           // 二进制指针
    wire [PTR_W-1:0] wptr_g, rptr_g;

    // 写操作
    always @(posedge wclk or negedge rst_n) begin
        if (!rst_n)     wptr <= 0;
        else if (winc && !full) begin
            mem[wptr[PTR_W-2:0]] <= wdata;
            wptr <= wptr + 1;
        end
    end
    // 读操作
    always @(posedge rclk or negedge rst_n) begin
        if (!rst_n)     rptr <= 0;
        else if (rinc && !empty)
            rptr <= rptr + 1;
    end
    assign rdata = mem[rptr[PTR_W-2:0]];

    // 格雷码转换
    assign wptr_g = wptr ^ (wptr >> 1);
    assign rptr_g = rptr ^ (rptr >> 1);

    // 指针跨域同步
    reg [PTR_W-1:0] wptr_gr1, wptr_gr2, rptr_gr1, rptr_gr2;
    always @(posedge rclk or negedge rst_n)
        if (!rst_n) {wptr_gr2,wptr_gr1} <= 0;
        else       {wptr_gr2,wptr_gr1} <= {wptr_gr1, wptr_g};
    always @(posedge wclk or negedge rst_n)
        if (!rst_n) {rptr_gr2,rptr_gr1} <= 0;
        else       {rptr_gr2,rptr_gr1} <= {rptr_gr1, rptr_g};

    // 空满判断（利用多出的1位）
    assign full  = (wptr_gr2 == {~rptr_g[PTR_W-1:PTR_W-2], rptr_g[PTR_W-3:0]});
```

```
    assign empty = (wptr_gr2 == rptr_gr2);
endmodule
```

面试提示：异步 FIFO 三大件——双口 RAM、格雷码指针跨域、空满判断。指针多 1 位（地址位+1）用于区分"空"（指针相等）和"满"（高两位相反）。能完整口述即可，不必逐行默写。

Q25：串并转换（高位优先 / 低位优先）

考点：来自原文档题目六。移位寄存器的典型应用。

思路：每拍将 1 bit 移入寄存器，计数满 8 拍输出。MSB first 存入 `data_o[7-cnt]`，LSB first 存入 `data_o[cnt]`。

代码答案（源自原文档 06-series_parallel.v，分两个独立模块避免多驱动）：

```
// ===== LSB First（低位先收）=====
module s2p_lsb (
    input clk, rst_n, data_i,
    output reg [7:0] data_o,
    output reg       done
);
    reg [2:0] cnt;
    always @(posedge clk or negedge rst_n) begin
        if (!rst_n) begin data_o <= 0; cnt <= 0; done <= 0; end
        else begin
            data_o[cnt] <= data_i;        // 先收的放低位
            cnt <= cnt + 1;
            done <= (cnt == 3'd7);
        end
    end
endmodule

// ===== MSB First（高位先收）=====
module s2p_msb (
    input clk, rst_n, data_i,
    output reg [7:0] data_o,
    output reg       done
);
    reg [2:0] cnt;
    always @(posedge clk or negedge rst_n) begin
        if (!rst_n) begin data_o <= 0; cnt <= 0; done <= 0; end
        else begin
            data_o[7-cnt] <= data_i;      // 先收的放高位
            cnt <= cnt + 1;
            done <= (cnt == 3'd7);
        end
    end
endmodule
```

面试提示：原文档将两个赋值写在同一个 `always` 块的两个分支里（对 `data_o` 双驱动，综合报错）。面试中写成**两个独立模块**或加 `mode` 参数选择更安全。

Q26：并串转换

考点：串并的逆操作。

代码答案：

```
module p2s (
    input clk, rst_n, load,
    input [7:0] par_in,
    output reg ser_out,
    output reg done
);
    reg [7:0] shift;
    reg [2:0] cnt;
    always @(posedge clk or negedge rst_n) begin
        if (!rst_n) begin shift <= 0; cnt <= 0; ser_out <= 0; done <= 0; end
        else if (load) begin shift <= par_in; cnt <= 0; done <= 0; end
        else begin
            ser_out <= shift[7];       // MSB 先出
            shift   <= shift << 1;
            cnt     <= cnt + 1;
            done    <= (cnt == 3'd7);
        end
    end
endmodule
```

Q27：同步 FIFO（单时钟）

考点：比异步 FIFO 简单，单时钟下的环形缓冲。

代码答案：

```
module sync_fifo #(
    parameter WIDTH = 8, DEPTH = 16
)(
    input clk, rst_n, winc, rinc,
    input  [WIDTH-1:0] wdata,
    output [WIDTH-1:0] rdata,
    output full, empty
);
    reg [WIDTH-1:0] mem [DEPTH-1:0];
    reg [$clog2(DEPTH):0] wptr, rptr;   // 多1位区分空满
    wire wfull  = (wptr[$clog2(DEPTH)] != rptr[$clog2(DEPTH)]) &&
                  (wptr[$clog2(DEPTH)-1:0] == rptr[$clog2(DEPTH)-1:0]);
    wire rempty = (wptr == rptr);

    always @(posedge clk or negedge rst_n) begin
        if (!rst_n) wptr <= 0;
        else if (winc && !wfull) begin mem[wptr[$clog2(DEPTH)-1:0]] <= wdata; wptr <= wptr+1; end
    end
    always @(posedge clk or negedge rst_n) begin
```

```
        if (!rst_n) rptr <= 0;
        else if (rinc && !rempty) rptr <= rptr+1;
    end
    assign rdata = mem[rptr[$clog2(DEPTH)-1:0]];
    assign full = wfull, empty = rempty;
endmodule
```

芯链团

第五篇 状态机与协议

Q28：摩尔型 vs 米利型状态机（以交通灯为例）

考点：FSM 基础概念 + 三段式写法。

代码答案（Moore 型三段式框架）：

```
module fsm_3seg (
    input clk, rst_n, start,
    output reg [1:0] out
);
    localparam S0=2'b00, S1=2'b01, S2=2'b10;
    reg [1:0] state, nstate;

    // 第一段：状态寄存
    always @(posedge clk or negedge rst_n)
        if (!rst_n) state <= S0; else state <= nstate;

    // 第二段：次态逻辑
    always @(*) begin
        nstate = state;
        case (state)
            S0: if (start) nstate = S1;
            S1: nstate = S2;
            S2: nstate = S0;
        endcase
    end

    // 第三段：输出逻辑（Moore：仅依赖状态）
    always @(*) begin
        case (state)
            S0: out = 2'b00;
            S1: out = 2'b01;
            S2: out = 2'b10;
        endcase
    end
endmodule
```

面试提示： - Moore：输出仅依赖当前状态（安全、无组合毛刺）；Mealy：输出依赖状态+输入（少1级延迟）。 - 三段式（状态寄存/次态/输出分离）是**最推荐写法**，易维护、综合友好。

Q29：可重叠序列检测 "1011"

考点：序列检测的进阶，考察重叠处理与状态化简。

思路：检测 1011。重叠情况如 `101011` 中 `1011` 出现一次； `1011011` 中第 2 个 1011 复用前者的末 1。状态数 = 模式长度+1 左右。

代码答案（Mealy 型，输入相关提前输出）：

```
module seq_1011 (
    input clk, rst_n, data,
    output reg det
);
    localparam S0=0,S1=1,S2=2,S3=3,S4=4;
    reg [2:0] state, nstate;
    always @(posedge clk or negedge rst_n)
        if (!rst_n) state <= S0; else state <= nstate;
    always @(*) begin
        nstate = S0; det = 0;
        case (state)
            S0: begin nstate = data?S1:S0; end
            S1: begin nstate = data?S1:S2; end
            S2: begin nstate = data?S3:S0; end
            S3: begin nstate = data?S1:S4; end   // 收1→有1011(输出1)，退回S1
            S4: begin nstate = data?S1:S2; det=1; end // 收1→1011(重叠)，收0→S2
        endcase
    end
endmodule
```

面试提示：重叠检测的关键——匹配后若收到能开启新模式的首字符（如 1011 后收 1，则"11"中后一个 1 是下一个 1011 的开头），需正确回到中间状态而非初始态。

Q30：实现握手协议（req / ack）

考点：AXI、APB 等总线的基础交互，也用于 CDC（见 Q21）。

代码答案（主从握手）：

```
module handshake_master (
    input clk, rst_n,
    input [7:0] data,
    output reg req,
    input ack,
    output reg done
);
    localparam IDLE=0, SEND=1, WAIT=2;
    reg [1:0] state;
    always @(posedge clk or negedge rst_n) begin
        if (!rst_n) begin state<=IDLE; req<=0; done<=0; end
        else case (state)
            IDLE: if (1) begin req<=1; state<=SEND; end   // 发起请求
            SEND: begin req<=1; state<=WAIT; end
            WAIT: if (ack) begin req<=0; done<=1; state<=IDLE; end  // 收到ack撤销
        endcase
    end
endmodule
```

Q31：PWM 发生器（占空比可调）

考点：电机控制、LED 调光常用。

代码答案：

```
module pwm #(
    parameter N = 8
)(
    input clk, rst_n,
    input [N-1:0] duty,   // 占空比 0~2^N-1
    output reg pwm_out
);
    reg [N-1:0] cnt;
    always @(posedge clk or negedge rst_n) begin
        if (!rst_n) cnt <= 0;
        else        cnt <= cnt + 1;
    end
    always @(posedge clk or negedge rst_n) begin
        if (!rst_n)   pwm_out <= 0;
        else          pwm_out <= (cnt < duty);  // 计数<占空比时输出高
    end
endmodule
```

面试提示：`cnt < duty` 比较实现占空比，duty 越大高电平占比越高。注意 cnt 回零时占空比为 0%（全低）。

Q32：脉宽检测（测量高电平持续周期数）

考点：测频、按键消抖、协议解析。

代码答案：

```
module pulse_width (
    input clk, rst_n,
    input sig,              // 待测信号
    output reg [15:0] width, // 高电平宽度（clk周期数）
    output reg done
);
    reg sig_d;
    reg measuring;
    always @(posedge clk or negedge rst_n) begin
        if (!rst_n) begin sig_d<=0; width<=0; done<=0; measuring<=0; end
        else begin
            sig_d <= sig;
            if (sig && !sig_d) begin       // 上升沿：开始
                measuring <= 1; width <= 0; done <= 0;
            end
            else if (sig && sig_d && measuring) width <= width + 1; // 计数
            else if (!sig && sig_d) begin    // 下降沿：结束
                measuring <= 0; done <= 1;
```

```
            end
        end
    end
endmodule
```

Q33：按键消抖（Debounce）

考点：FPGA 开发中极常见，考察边沿+计数。

代码答案：

```
module debounce #(
    parameter CNT_MAX = 1000000  // 约20ms@50MHz
)(
    input clk, rst_n, btn_in,
    output reg btn_out
);
    reg btn_d;
    reg [$clog2(CNT_MAX)-1:0] cnt;
    wire stable = (btn_in == btn_d);
    always @(posedge clk or negedge rst_n) begin
        if (!rst_n) begin btn_d<=0; cnt<=0; btn_out<=0; end
        else begin
            btn_d <= btn_in;
            if (!stable) cnt <= 0;                  // 仍在抖动，清零
            else if (cnt < CNT_MAX) cnt <= cnt + 1;   // 稳定计时
            else btn_out <= btn_d;                    // 稳定足够久才输出
        end
    end
endmodule
```

芯链团

第六篇 进阶与综合

Q34：实现线性反馈移位寄存器（LFSR，伪随机数）

考点：伪随机序列、BIST、加扰。

代码答案（4 阶 Fibonacci LFSR，本原多项式 x^4+x^3+1）：

```
module lfsr4 (
    input clk, rst_n, en,
    output reg [3:0] lfsr
);
    wire feedback = lfsr[3] ^ lfsr[2];   // 抽头：bit3 与 bit2
    always @(posedge clk or negedge rst_n) begin
        if (!rst_n) lfsr <= 4'b0001;      // 初值非全0
        else if (en) lfsr <= {lfsr[2:0], feedback};
    end
endmodule
```

面试提示：LFSR 初值不能为全 0（会锁死）。抽头选择决定序列周期（最大为 2^n-1）。

Q35：实现一个简单仲裁器（固定优先级 / 轮询）

考点：多主总线、DMA 请求调度。

代码答案（固定优先级仲裁）：

```
module arb_fixed (
    input [3:0] req,
    input [1:0] grant_prev,   // 上一拍授权（用于轮询）
    output reg [1:0] grant
);
    always @(*) begin
        case (1'b1)
            req[0]: grant = 2'd0;
            req[1]: grant = 2'd1;
            req[2]: grant = 2'd2;
            req[3]: grant = 2'd3;
            default: grant = 2'd0;
        endcase
    end
endmodule
```

Q36：CRC 校验（以 CRC-8 为例）

考点：通信可靠性，考察移位+异或的串行处理。

代码答案（CRC-8 多项式 0x07，串行逐 bit）：

```
module crc8 (
    input clk, rst_n, data_in, data_valid,
    output reg [7:0] crc,
    output reg       crc_valid
);
    parameter POLY = 8'h07;
    reg [7:0] crc_reg;
    reg [2:0] bit_cnt;
    always @(posedge clk or negedge rst_n) begin
        if (!rst_n) begin crc_reg<=0; bit_cnt<=0; crc_valid<=0; end
        else if (data_valid) begin
            // 每 byte 8 bit 逐位计算
            crc_reg <= (crc_reg[0] ^ data_in) ?
                       {1'b0, crc_reg[7:1]} ^ POLY :
                       {1'b0, crc_reg[7:1]};
            bit_cnt <= bit_cnt + 1;
            crc_valid <= (bit_cnt == 3'd7);
        end
    end
    assign crc = crc_reg;
endmodule
```

面试提示：CRC 实现有串行（逐 bit，慢）和并行（查表/组合逻辑，快）两种。面试能讲清 LFSR 思想即可。

Q37：实现 7 段数码管动态扫描显示

考点：FPGA 外设驱动，考察分时复用。

代码答案：

```
module seg_scan #(
    parameter N = 4   // 数码管位数
)(
    input clk, rst_n,
    input [N*4-1:0] bcd,    // 每个数码管4位BCD
    output reg [N-1:0] sel, // 位选（低有效）
    output reg [6:0] seg    // 段选 a~g
);
    reg [15:0] cnt;
    reg [1:0]  digit;
    always @(posedge clk or negedge rst_n)
        if (!rst_n) cnt <= 0; else cnt <= cnt + 1;

    always @(posedge clk or negedge rst_n) begin
        if (!rst_n) digit <= 0;
        else if (cnt == 0) digit <= digit + 1;
    end

    // 位选
    always @(*) begin
```

```
        sel = ~(1 << digit);
        case (digit)
            0: seg = bcd[3:0];
            1: seg = bcd[7:4];
            2: seg = bcd[11:8];
            3: seg = bcd[15:12];
        endcase
    end

    // BCD→7段译码
    function [6:0] dec2seg;
        input [3:0] b;
        case (b)
            0: dec2seg = 7'b1111110;
            1: dec2seg = 7'b0110000;
            2: dec2seg = 7'b1101101;
            3: dec2seg = 7'b1111001;
            4: dec2seg = 7'b0110011;
            5: dec2seg = 7'b1011011;
            6: dec2seg = 7'b1011111;
            7: dec2seg = 7'b1110000;
            8: dec2seg = 7'b1111111;
            9: dec2seg = 7'b1111011;
            default: dec2seg = 7'b0000000;
        endcase
    endfunction
endmodule
```

Q38：查找表实现 sin/cos（CORDIC 简化）

考点：DSP 算法硬件实现入门。

代码答案（LUT 查表法）：

```
module sin_lut (
    input  [7:0] addr,    // 相位 0~255
    output [7:0] sin_out // 有符号 -128~127
);
    reg [7:0] rom [0:255];
    integer i;
    initial begin
        for (i = 0; i < 256; i = i + 1)
            rom[i] = $sin(i/256.0*3.14159*2) * 127;
    end
    assign sin_out = rom[addr];
endmodule
```

面试提示：`initial` 块中的 `for` 给 ROM 赋初值在 FPGA 综合中合法（推断为初始化块 ROM），但在 ASIC 中需用复位或外部加载。

Q39：实现二分频的另一种写法 + 任意奇数分频通用模板

考点：分频综合，考察参数化思维。

代码答案（通用奇数 N 分频 50% 占空比模板）：

```
module clk_div_odd #(
    parameter N = 5    // 奇数，如 3/5/7
)(
    input clk, rst_n,
    output clk_out
);
    reg [log2(N)-1:0] cnt;
    reg p, n;
    always @(posedge clk or negedge rst_n) begin
        if (!rst_n) cnt <= 0;
        else cnt <= (cnt == N-1) ? 0 : cnt + 1;
    end
    // 上升沿：cnt < (N+1)/2 时高
    always @(posedge clk or negedge rst_n)
        if (!rst_n) p <= 0; else p <= (cnt < (N+1)/2);
    // 下降沿：同样区间高（错相位）
    always @(negedge clk or negedge rst_n)
        if (!rst_n) n <= 0; else n <= (cnt < (N+1)/2);
    assign clk_out = p | n;
endmodule
```

Q40：综合题——用状态机实现一个简单的 UART 接收机（8N1）

考点：综合应用：状态机 + 边沿检测 + 计数器 + 移位寄存器。压轴题。

代码答案（核心接收状态机）：

```
module uart_rx (
    input clk, rst_n, rx,       // rx 为接收引脚
    output reg [7:0] data_out,
    output reg       data_valid
);
    parameter BAUD_CNT = 16;     // 每 bit 16 个 clk（示意，实际按波特率算）
    localparam IDLE=0, START=1, DATA=2, STOP=3;
    reg [1:0] state;
    reg [3:0] bit_cnt;
    reg [3:0] baud_cnt;
    reg       rx_d;
    wire rx_fall = rx_d & ~rx;    // 下降沿 = 起始位

    always @(posedge clk or negedge rst_n) begin
        if (!rst_n) begin state<=IDLE; rx_d<=0; bit_cnt<=0; baud_cnt<=0; data_valid<=0; end
        else begin
            rx_d <= rx;
            case (state)
```

```
            IDLE: begin
                data_valid <= 0;
                if (rx_fall) begin state <= START; baud_cnt <= 0; end
            end
            START: begin  // 等半位宽后采样确认起始位
                if (baud_cnt == BAUD_CNT/2-1) begin baud_cnt<=0; state<=DATA; bit_cnt<=0; end
                else baud_cnt <= baud_cnt + 1;
            end
            DATA: begin
                if (baud_cnt == BAUD_CNT-1) begin
                    baud_cnt <= 0;
                    data_out[bit_cnt] <= rx;     // 在 bit 中间采样
                    if (bit_cnt == 7) begin bit_cnt<=0; state<=STOP; end
                    else bit_cnt <= bit_cnt + 1;
                end
                else baud_cnt <= baud_cnt + 1;
            end
            STOP: begin
                if (baud_cnt == BAUD_CNT-1) begin
                    baud_cnt <= 0; data_valid <= 1; state <= IDLE;
                end
                else baud_cnt <= baud_cnt + 1;
            end
        endcase
    end
  end
endmodule
```

面试提示：UART 接收核心是"起始位下降沿触发 → 半位宽后开始按位采样 → 8 个数据位 → 停止位"。采样点选在每位中间（baud_cnt == BAUD_CNT/2）以规避边沿抖动。

附录 A：手撕代码通用检查清单

检查项	是否通过
接口含 `clk` / `rst_n`	☐
复位风格统一（异步低有效 / 同步）	☐
组合逻辑用 `always @(*)` 且 `case` 有 `default`	☐
时序逻辑用非阻塞 `<=`，组合用阻塞 `=`	☐
计数器初值/边界正确（`< N` vs `<= N-1`）	☐
状态机三段式（状态寄存/次态/输出分离）	☐
CDC 单比特打拍、多比特握手/FIFO	☐
无 `initial` 非 0 初值（ASIC 不可综合）	☐
无 `#delay`（不可综合）	☐
位宽匹配（避免隐式截断/扩展）	☐

附录 B：常考频率排序（面试命中率参考）

排名	题目	出现频率
1	序列检测（101/1011）	☆☆☆☆☆
2	边沿检测	☆☆☆☆☆
3	分频（奇/偶）	☆☆☆☆☆
4	异步 FIFO	☆☆☆☆
5	跨时钟域同步	☆☆☆☆
6	状态机（三段式）	☆☆☆☆
7	串并/并串转换	☆☆☆☆
8	时钟切换（无毛刺）	☆☆☆
9	计数器/移位寄存器	☆☆☆
10	握手/仲裁/CRC/LFSR	☆☆☆

> **芯链团** | 本资料由 AI 辅助生成，基于公开面经与《FPGA 手撕代码经典 8 道题目》源码整理扩展。所有代码均按可综合　RTL　规范编写，但面试答题时请结合目标公司的编码规范与具体约束。重要设计决策（如复位策略、CDC 方案）请以实际项目要求为准。

芯链团